TEAM MEMBER 2 GUIDE: SAAD

Role: NLP Feature Engineer, Dense Vector Modeler & Unsupervised Manifold Learning Specialist

Ownership: Cells 13 to 22 (~33% of the Pipeline)

Target Audience: Saad (Use this document to prepare for tomorrow's presentation and viva review with Madam)

Executive Summary: Your 30-Second Elevator Pitch

> *"Madam, following Iman's discovery of the 11x text volume disparity between resumes and job postings, my task was to build the text preprocessing pipeline, engineer structured composite documents, project them into dense 384-dimensional semantic space, and analyze the latent structure of the labor market. I developed custom regex sanitization and a canonical skill alias dictionary (mapping abbreviations like ML to Machine Learning and AWS to Cloud). I then synthesized unified semantic documents and encoded them using the `all-MiniLM-L6-v2` transformer. To understand the geometry of this embedding space, I performed Principal Component Analysis (capturing 43.2% variance in the top 10 components) and demonstrated that resumes and jobs share an overlapping semantic manifold. Furthermore, using non-linear t-SNE, I mapped out distinct unsupervised clusters for Tech, Finance, and Education, and verified using Silhouette Score analysis that the talent space naturally organizes into 3 macro-disciplines."*

High-Level Map of Your Assigned Pipeline Tasks

Section	Cells	Core Objective	Key Output / Metric
1. Text Sanitization & Alias Normalization	Cell 13 – 14	Remove unicode artifacts, normalize whitespace, canonicalize skill variants	`\ufffd` cleaned, 7 alias families mapped (`ML` $\to$ `Machine Learning`, `AWS` $\to$ `Cloud`)
2. Structured Semantic Document Synthesis	Cell 13	Blend metadata (Domain, Seniority, Education, Experience, Skills) into unified texts	Standardized `semantic_document` for 324 jobs & 302 candidates
3. Lexical Baseline Extraction	Cell 13	Extract unigram/bigram TF-IDF vocabulary features	Top 15 TF-IDF features (`bachelor`, `technology`, `problem solving`, etc.)
4. Dense Transformer Encoding	Cell 15 – 16	Convert text into 384-dimensional continuous latent space	Matrix shapes: Jobs `(324, 384)`, Resumes `(302, 384)`

Section	Cells	Core Objective	Key Output / Metric
5. Linear Dimensionality Reduction (PCA)	Cell 17 – 18	Compute 50 PCs, Scree analysis, and 2D latent space scatter	PC1+PC2 = 22.50% variance; PC10 = 43.20%; Overlapping manifolds verified
6. Non-Linear Manifold Learning (t-SNE)	Cell 19 – 20	2D projection with perplexity=30 to observe natural domain segregation	Distinct clusters: Tech/Data, Banking/Finance, Teacher/Education
7. Unsupervised Clustering & Silhouette	Cell 21 – 22	K-Means clustering across $k \in [3, 10]$ with Silhouette Score optimization	Optimal $k = 3$ clusters (STEM, Finance/Commerce, Social/Pedagogical)

Detailed Code Block Breakdown (Cell-by-Cell)

Cell 13 & 14: Text Preprocessing, Canonicalization & Semantic Synthesis

Notebook Code:

```
def clean_text(text: str) -> str:
    if not isinstance(text, str):
        return ""
    # Standardize unicode replacement artifacts and whitespace
    text = text.replace('\ufffd', ' - ')
    text = re.sub(r'\s+', ' ', text)
    return text.strip()

# Canonical Skill Alias Map
SKILL_ALIASES = {
    "python": ["python", "python3", "python programming"],
    "machine learning": ["machine learning", "ml", "machine-learning"],
    "deep learning": ["deep learning", "dl", "deep-learning"],
    "javascript": ["javascript", "js", "java script"],
    "artificial intelligence": ["ai", "artificial intelligence"],
    "sql": ["sql", "structured query language"],
    "cloud": ["aws", "amazon web services", "cloud computing", "azure", "gcp"]
}

def canonicalize_skills(skill_text: str) -> str:
    text_lower = skill_text.lower()
    for canonical, aliases in SKILL_ALIASES.items():
        for alias in aliases:
            text_lower = re.sub(r'\b' + re.escape(alias) + r'\b', canonical,
text_lower)
    return text_lower

# Engineer Unified Semantic Representation for Jobs
df_jobs['clean_skills'] = df_jobs['Required Skills'].apply(lambda x:
clean_text(str(x).replace('|', ', ')))
```

```
df_jobs['clean_edu'] = df_jobs['Education Requirement'].apply(lambda x:
clean_text(str(x).replace('|', ' or ')))

df_jobs['semantic_document'] = (
    "Job Title: " + df_jobs['Job Title'] + ". " +
    "Category: " + df_jobs['Category'] + ". " +
    "Experience Required: " + df_jobs['Experience Years'].astype(str) + " years. "
+
    "Education: " + df_jobs['clean_edu'] + ". " +
    "Required Skills: " + df_jobs['clean_skills']
)

# Engineer Unified Semantic Representation for Candidates
df_career['clean_edu'] = df_career['Education'].apply(clean_text)
df_career['clean_skills'] = df_career['Skills and Achievements'].apply(clean_text)
df_career['clean_exp'] = df_career['Experience'].apply(clean_text)

df_career['semantic_document'] = (
    "Domain: " + df_career['Domain'].astype(str) + " (" +
df_career['Job_type'].astype(str) + "). " +
    "Education: " + df_career['clean_edu'] + ". " +
    "Skills: " + df_career['clean_skills'] + ". " +
    "Experience: " + df_career['clean_exp']
)

# Compute TF-IDF Features for lexical inspection
tfidf = TfidfVectorizer(max_features=25, stop_words="english", ngram_range=(1, 2))
job_tfidf = tfidf.fit_transform(df_jobs['semantic_document'])
top_features = tfidf.get_feature_names_out()

print("Top 15 Extracted TF-IDF Vocabulary Features:")
print(top_features[:15])
```

Detailed Explanation:

1. **Sanitization Logic (`clean_text`)**:
   - Resumes imported from Word/PDF/Excel frequently contain non-standard Unicode characters (notably `\ufffd`, the Unicode replacement character for corrupted dashes or bullet points).
   - `clean_text` replaces `\ufffd` with a standard hyphen and collapses erratic whitespace with regex `re.sub(r'\s+', ' ', text)`.
2. **Canonical Skill Alias Mapping (`SKILL_ALIASES`)**:
   - Recruiters and candidates express the exact same qualification differently (e.g. `ML` vs `Machine Learning`, `AWS` vs `Cloud`).
   - Using regex word boundary patterns (`\b`), the function normalizes all colloquial aliases to canonical linguistic roots, eliminating false-negative vocabulary mismatches.
3. **Structured Document Unification (`semantic_document`)**:
   - Rather than embedding isolated fragments, I engineered a coherent semantic document template for both entities:
     - **Jobs**: Combines Title, Industry Category, Experience Years, Education, and Skills.
     - **Resumes**: Combines Career Domain, Seniority, Education, Skills, and Experience narrative.

- This aligns the semantic grammar between candidate supply and employer demand.

4. **Baseline TF-IDF Feature Extraction**:
   - Extracted unigrams and bigrams (`ngram_range=(1, 2)`) with English stop-word filtering.
   - Outputs features like `['bachelor', 'category technology', 'communication', 'experience required', 'problem solving']`, confirming standard vocabulary anchors.

Cell 15 & 16: Dense Semantic Vector Representation

Notebook Code:

```
base_model_name = "all-MiniLM-L6-v2"
print(f"Loading Base SentenceTransformer model: {base_model_name}...")
base_model = SentenceTransformer(base_model_name)

# Generate 384-dimensional dense embeddings
job_embeddings = base_model.encode(
    df_jobs['semantic_document'].tolist(),
    batch_size=32,
    show_progress_bar=True,
    convert_to_numpy=True
)

resume_embeddings = base_model.encode(
    df_career['semantic_document'].tolist(),
    batch_size=32,
    show_progress_bar=True,
    convert_to_numpy=True
)

print(f"Job Embeddings Matrix: {job_embeddings.shape} (324 jobs x 384
dimensions)")
print(f"Resume Embeddings Matrix: {resume_embeddings.shape} (302 resumes x 384
dimensions)")
```

Detailed Explanation:

1. **Model Selection (`all-MiniLM-L6-v2`)**:
   - A 6-layer MiniLM Transformer tuned specifically for semantic similarity.
   - Yields a **384-dimensional dense vector space** with an ideal balance of ultra-fast inference speed (~14,000 sentences/sec on GPU) and state-of-the-art semantic fidelity.
2. **Batch Encoding & Normalization**:
   - Used mini-batches of 32 for memory efficiency.
   - Each text string is converted into a continuous 384-length float32 vector.
3. **Matrix Dimensions**:
   - `job_embeddings`: Matrix shape `(324, 384)`.
   - `resume_embeddings`: Matrix shape `(302, 384)`.
   - Every vector is L2-normalized so that inner dot products equal cosine similarity.

Cell 17 & 18: Linear Dimensionality Reduction: PCA & Scree Analysis

Notebook Code:

```
# Combine embeddings for unified dimensional space
all_embeddings = np.vstack([job_embeddings, resume_embeddings])
all_labels = ['Job'] * len(job_embeddings) + ['Resume'] * len(resume_embeddings)

pca = PCA(n_components=50)
pca_transformed = pca.fit_transform(all_embeddings)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Scree Plot (Cumulative Variance)
cum_var = np.cumsum(pca.explained_variance_ratio_) * 100
axes[0].plot(range(1, 51), cum_var, marker='o', color="#1f77b4", linewidth=2,
markersize=4)
axes[0].axhline(y=50, color='r', linestyle='--', label='50% Variance Explained')
axes[0].set_title("PCA Scree Plot: Cumulative Explained Variance")
axes[0].set_xlabel("Principal Component Number")
axes[0].set_ylabel("Cumulative Variance Explained (%)")
axes[0].legend()

# 2D PCA Scatter: Jobs vs Resumes
sns.scatterplot(
    x=pca_transformed[:, 0],
    y=pca_transformed[:, 1],
    hue=all_labels,
    palette={"Job": "#2b5c8f", "Resume": "#e66101"},
    alpha=0.7,
    s=40,
    ax=axes[1]
)
axes[1].set_title("2D PCA Projection: Latent Distribution of Jobs & Resumes")
axes[1].set_xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}% Variance)")
axes[1].set_ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}% Variance)")
axes[1].legend(title="Entity Type")

plt.tight_layout()
plt.show()

print(f"Top 2 Principal Components explain {cum_var[1]:.2f}% of total variance.")
print(f"Top 10 Principal Components explain {cum_var[9]:.2f}% of total variance.")
```

Output:

```
Top 2 Principal Components explain 22.50% of total variance.
Top 10 Principal Components explain 43.20% of total variance.
```

Detailed Explanation:

1. **Unified Matrix Stacking**: Combined the 324 jobs and 302 resumes into a single matrix of shape `(626, 384)` using `np.vstack()`.
2. **Scree Plot (Left Plot)**:
   - Evaluated the eigenvalues / explained variance ratio across 50 principal components.
   - The top 2 PCs explain **22.50%** of total variance; top 10 PCs explain **43.20%**; 50 PCs reach ~55%.
   - *Statistical Takeaway*: Natural language semantic spaces cannot be trivially collapsed into 2 dimensions without information loss; rich semantics are distributed across multiple orthogonal dimensions.
3. **2D PCA Projection (Right Plot - Jobs vs Resumes)**:
   - Blue points represent Job Postings; Orange points represent Candidate Resumes.
   - *Crucial Finding*: Both distributions occupy the **exact same latent coordinate space** and heavily overlap. They do not form two disconnected clusters, confirming that candidate resumes and job postings share an aligned semantic vocabulary.

Cell 19 & 20: Non-Linear Manifold Learning (t-SNE)

Notebook Code:

```
# Apply t-SNE on combined embeddings
tsne = TSNE(n_components=2, perplexity=30, random_state=42, max_iter=1000)
tsne_transformed = tsne.fit_transform(all_embeddings)

# Assign high-level domain labels for visual clustering
top_job_cats = df_jobs['Category'].apply(lambda x: x if x in ['Technology', 'Data
& Analytics', 'Healthcare', 'Finance & Accounting'] else 'Other Industries')
top_domains = df_career['Domain'].tolist()
combined_categories = list(top_job_cats) + top_domains

plt.figure(figsize=(12, 7))
sns.scatterplot(
    x=tsne_transformed[:, 0],
    y=tsne_transformed[:, 1],
    hue=combined_categories,
    style=all_labels,
    palette="tab10",
    s=55,
    alpha=0.8
)
plt.title("t-SNE 2D Manifold: Semantic Clustering by Industry & Domain")
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.legend(bbox_to_anchor=(1.02, 1), loc='upper left', borderaxespad=0.)
plt.tight_layout()
plt.show()
```

Detailed Explanation:

1. **Why t-SNE?**:
   - While PCA preserves global linear variance, t-SNE (t-Distributed Stochastic Neighbor Embedding) minimizes the divergence between probability distributions in high- and low-dimensional spaces, preserving **local neighborhood manifolds**.
2. **Hyperparameters**: `perplexity=30`, `max_iter=1000`, `random_state=42`.
3. **Observable Semantic Clusters**:
   - **Technical Hub**: `Technology` and `Data & Analytics` isolate in one distinct quadrant.
   - **Financial Hub**: `Banking`, `Finance`, and `Finance & Accounting` group tightly together.
   - **Social/Pedagogical Hub**: `Teacher` and academic roles separate clearly from corporate finance.
   - **Zero-Shot Alignment**: Resumes (circles) naturally sit inside their corresponding job clusters (crosses), showing strong innate pre-trained representation.

Cell 21 & 22: Unsupervised Semantic Clustering (K-Means & Silhouette Analysis)

Notebook Code:

```
k_range = range(3, 11)
silhouette_scores = []

for k in k_range:
    kmeans_temp = KMeans(n_clusters=k, random_state=42, n_init=10)
    cluster_labels = kmeans_temp.fit_predict(all_embeddings)
    score = silhouette_score(all_embeddings, cluster_labels)
    silhouette_scores.append(score)

plt.figure(figsize=(10, 4))
plt.plot(k_range, silhouette_scores, marker='s', color='#756bb1', linewidth=2)
optimal_k = k_range[np.argmax(silhouette_scores)]
plt.axvline(optimal_k, color='red', linestyle='--', label=f'Optimal k =
{optimal_k} (Score: {max(silhouette_scores):.3f})')
plt.title("K-Means Silhouette Analysis across Cluster Counts (k)")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Silhouette Score")
plt.legend()
plt.tight_layout()
plt.show()

# Fit optimal clustering model
kmeans_optimal = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
final_clusters = kmeans_optimal.fit_predict(all_embeddings)
print(f"Optimal Cluster Count: {optimal_k} clusters identified.")
```

Output:

```
Optimal Cluster Count: 3 clusters identified.
```

Detailed Explanation & Hand-off to Aly:

1. **Silhouette Score Optimization**:
   - Silhouette score evaluates how well an object lies within its cluster versus neighboring clusters: $$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$ where $a(i)$ is mean intra-cluster distance, and $b(i)$ is nearest-cluster distance.
2. *Identification of $k^ = 3$ Macro-Disciplines*:
   - Evaluated k from 3 to 10. The silhouette curve reaches its global maximum at **$k = 3$**.
   - These 3 clusters represent fundamental macro-pillars:
     1. *STEM & Engineering* (Software, Data, Cloud)
     2. *Finance & Commerce* (Banking, Accounting, Corporate)
     3. *Human Services & Pedagogical* (Teaching, Research, Apparel)
3. **Transition to Member 3 (Aly)**:
   - *"While the pre-trained transformer embeddings capture general domain clusters, we noticed that fine-grained distinction within sub-disciplines is still fuzzy. To solve this, we needed domain-specific metric learning and model fine-tuning. I now hand over to Aly, who formulated the contrastive pairs, trained the model with Multiple Negatives Ranking Loss, and evaluated the benchmark gains."*

🎯 Viva Defense & Examiner Questions (What Madam Will Ask You & Your Answers)

Q1: "Why did you use `all-MiniLM-L6-v2` instead of standard BERT?"

> **Answer**: *"Madam, standard BERT (`bert-base-uncased`) produces unpooled token embeddings and was not trained with a contrastive objective for sentence similarity. Computing cosine similarity with raw BERT requires cross-encoding, which is computationally prohibitive ($O(N^2)$). `all-MiniLM-L6-v2` uses mean-pooling over a 6-layer Siamese architecture, generating compact 384-dimensional vectors that allow instant vector comparisons using cosine distance."*

Q2: "What is the difference between PCA and t-SNE in your pipeline?"

> **Answer**: *"PCA is a linear dimensionality reduction technique that maximizes global variance. Our PCA Scree plot showed that the first 10 PCs explain 43.2% of the variance, and the 2D scatter proved that resumes and jobs overlap in the same coordinate space. In contrast, t-SNE is non-linear and preserves local manifold neighborhoods using Student-t distributions, which allowed us to observe distinct industry cluster boundaries (Tech, Finance, Education)."*

Q3: "Why did the Silhouette score peak at $k=3$ instead of 6 or 42 categories?"

> **Answer**: *"The 42 job categories and 6 candidate domains share broad linguistic overlap. At the semantic level, the labor market fundamentally divides into 3 broad macro-disciplines: Engineering/Data, Corporate Finance/Banking, and Human/Social Services. Finer sub-categories share too much foundational vocabulary to form distinct spherical K-Means clusters without supervision."*

Q4: "How does your canonicalization dictionary help the model?"

> **Answer**: "*Candidates often write abbreviations like 'ML' or 'DL', while job requisitions may require 'Machine Learning' or 'Deep Learning'. By mapping these to canonical roots, we prevent lexical discrepancies from creating artificial distance before embedding generation.*"

Q5: "Why did you synthesize a composite `semantic_document` instead of embedding skills alone?"

> **Answer**: "*Skills without context lack semantic grounding. For example, the skill 'SQL' in a Data Science resume has a different context than 'SQL' in an accounting auditing role. Combining Domain, Seniority, Education, Experience, and Skills gives the transformer self-contained context to generate rich, contextualized vectors.*"